

Advanced Topics in Computer Graphics I

Ordinary Monte Carlo Light Transport Simulation



Prof. Dr. Reinhard Klein

Institute of Computer Science II - Computer Graphics
University of Bonn

June 1, 2026



- ▶ Ordinary Monte Carlo (OMC)
- ▶ Photon Mapping
- ▶ Markov Chain Monte Carlo (MCMC)



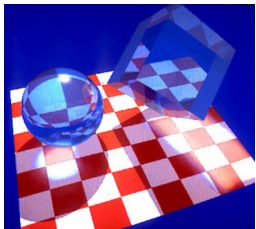
Ordinary Monte Carlo Methods



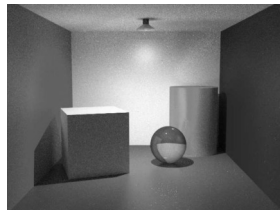
► Path tracing, Kajiya, 1986



► Light tracing, Arvo, 1986, Dutre et al., 1993



► Bidirectional Path Tracing, Lafortune and Willems, 1993



abcd

^aJ. T. Kajiya. "The rendering equation". In: *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*. 1986, pp. 143–150.

^bJ. Arvo et al. "Backward ray tracing". In: *Developments in ray tracing 12* (1986), pp. 259–263.

^cP. Dutré et al. "Monte Carlo light tracing with direct computation of pixel intensities". In: (1993).

^dE. P. Lafortune and Y. D. Willems. "Bi-directional path tracing". In: (1993).



We introduced some important Monte Carlo techniques. Now, we put everything together to the **Monte-Carlo Ray Tracing** or **Path Tracing** algorithm (Kajiya, 1986).

According to the Path-Space Measurement Equation the incident irradiance at pixel j is given by the contribution of all possible pathes of length $1, 2, \dots$:

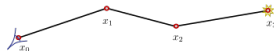
$$I_j = \sum_{i=1}^{\infty} \underbrace{\int_S \cdots \int_S}_{i+1} W_e^{(j)}(x_0 \rightarrow x_1) T(\bar{x}_i) L_e(x_{i+1} \rightarrow x_i) G(x_0 \leftrightarrow x_1) dS(x_{i+1}) \cdots dS(x_0),$$

The basic idea of **Path Tracing** is to estimate each of the individual terms, i.e. the high-dimensional $2(i+1)$ integrals in this sum, by the contribution of **only one path** of length i between the light source and pixel j .

For this purpose a ray is traced starting from a location x_0 on pixel j into an arbitrary direction which is chosen according to a probability density p that depends on x_0 . At each intersection point $x_k, k = 1, \dots, i$ of the ray with the scene geometry, the path is continued by choosing a novel direction according to probability densities which depend on the intersection points x_k .



Reasoning: This way the contribution of all pathes of length, e.g. 3, is estimated using **one** sample per two-dimensional integral:



$$\begin{aligned}
 & \int_S \int_S \int_S \int_S W_e^{(j)}(x_0 \rightarrow x_1) G(x_0 \leftrightarrow x_1) \rho(x_1) G(x_1 \leftrightarrow x_2) \rho(x_2) G(x_2 \leftrightarrow x_3) L_e(x_3 \rightarrow x_2) dS(x_3) \cdots dS(x_0), \\
 &= \int_S \int_S \int_S \frac{W_e^{(j)}(\hat{x}_0 \rightarrow x_1)}{p(\hat{x}_0)} G(\hat{x}_0 \leftrightarrow x_1) \rho(x_1) G(x_1 \leftrightarrow x_2) \rho(x_2) G(x_2 \leftrightarrow x_3) L_e(x_3 \rightarrow x_2) dS(x_3) dS(x_2) dS(x_1), \\
 &= \int_S \int_S \frac{W_e^{(j)}(\hat{x}_0 \rightarrow \hat{x}_1)}{p(\hat{x}_0)} \frac{G(\hat{x}_0 \leftrightarrow \hat{x}_1) \rho(\hat{x}_1)}{p(\hat{x}_1)} G(\hat{x}_1 \leftrightarrow x_2) \rho(x_2) G(x_2 \leftrightarrow x_3) L_e(x_3 \rightarrow x_2) dS(x_3) dS(x_2), \\
 &= \dots \\
 &= \frac{W_e^{(j)}(\hat{x}_0 \rightarrow \hat{x}_1)}{p(\hat{x}_0)} \frac{G(\hat{x}_0 \leftrightarrow \hat{x}_1) \rho(\hat{x}_1)}{p(\hat{x}_1)} \frac{G(\hat{x}_1 \leftrightarrow \hat{x}_2) \rho(\hat{x}_2)}{p(\hat{x}_2)} \frac{G(\hat{x}_2 \leftrightarrow \hat{x}_3) L_e(\hat{x}_3 \rightarrow \hat{x}_2)}{p(\hat{x}_3)},
 \end{aligned}$$

where $\rho_s(x_i) = \rho_s(x_{i-1} \rightarrow x_i \rightarrow x_{i+1})$ and $p(x_i)$ is the probability for choosing the point x_i .



- ▶ By constructing a **single path** from the lens to a light source, the $2i + 1$ dimensional integral is estimated by the amount of light transported along $\bar{x}_i = x_0, \dots, x_i$.
- ▶ The probability of the path is given by the joint distribution of the individual path vertices $x_0, \dots, x_i$ which is the product of their conditional pdfs $p(x_0, \dots, x_i) = p(x_0) \cdots p(x_i)$.
- ▶ This way the path integral formulation enables the construction of an estimator for a pixel value by first sampling a random path $\bar{x}_i$ connecting the eye with a light source and then evaluating $\frac{f(\bar{x}_i)}{p(\bar{x}_i)}$, where

$$f(\bar{x}_i) = W_e^{(j)}(x_0 \rightarrow x_1)G(x_0 \leftrightarrow x_1)\rho(x_1)G(x_1 \leftrightarrow x_2) \cdots \rho(x_{i-1})G(x_{i-1} \leftrightarrow x_i)L_e(x_i)$$

- ▶ Different path sampling techniques can be combined leading to different variants of the algorithm.



How to compute the probability of a path?

- ▶ Straight forward is to use a uniform distribution of the points on the surface: sample vertices according to the surface area of objects in the scene. For this we define a discrete probability over the n objects in the scene. If each has surface area A_i , then the probability of sampling a path vertex on the surface of the i th object should be $p_i = \frac{A_i}{\sum_j A_j}$. Then, given a method to sample a point on the i th object with uniform probability, the PDF for sampling any particular point p_k on object is $\frac{1}{A_i}$. We get

$$p_A(p_k) = \frac{A_i}{\sum_j A_j} \frac{1}{A_i} = \frac{1}{\sum_j A_j}.$$

- ▶ Visibility is not considered and many of the paths will have no contribution if they have pairs of adjacent vertices that are not mutually visible.
- ▶ If the integrand has functions in it that are close to a delta function like small light sources, point light sources or a nearly perfectly specular BSDF, the resulting path vertices will hardly be able to capture these distributions resulting in a high variance of the estimator.
- ▶ Therefore, in practice, BSDF- and light source sampling techniques are applied.



Goal: Choose outgoing directions $\hat{\omega}$ with PDF proportional to $f_{BRDF} \cdot \cos(\theta)$ to minimise variance of the path estimator.

Lambertian (diffuse) surfaces: $f_{BRDF} = \rho/\pi$. The ideal PDF is

$$p(\hat{\omega}) = \frac{\cos(\theta)}{\pi}.$$

This is achieved by *cosine-weighted hemisphere sampling*. The per-sample weight then simplifies:

$$\frac{f_{BRDF}(\hat{\omega}, \mathbf{x}, \boldsymbol{\omega}_o) \cos(\theta)}{p(\hat{\omega})} = \frac{(\rho/\pi) \cos(\theta)}{(\cos(\theta)/\pi)} = \rho.$$

The $\cos(\theta)$ and π factors cancel exactly — the estimator contribution reduces to the **albedo** ρ times the recursively traced radiance.

Glossy / specular surfaces: Sample proportional to the BRDF lobe (e.g. using the GGX distribution for microfacet models). As the lobe narrows, fewer sampled directions reach the light source.

Key insight

BSDF sampling is efficient when the illumination is *smooth and broad*. It fails for small or localised light sources, where the sampled directions rarely hit any emitter.



The **path throughput** β accumulates BSDF, cosine, and PDF factors incrementally along a path, avoiding recomputation of the full product at every bounce:

$$\beta_0 = \mathbf{W}_e^{(j)}, \quad \beta_{k+1} = \beta_k \cdot \frac{f_{BRDF}(\hat{\omega}_{k+1}, \mathbf{x}_k, \boldsymbol{\omega}_k) \cos(\hat{\theta}_{k+1})}{p(\mathbf{x}_k, \hat{\omega}_{k+1})}.$$

The MC estimator for a path of length i is then:

$$\frac{f(\bar{x}_i)}{p(\bar{x}_i)} = \beta_i \cdot \mathbf{L}_e(\mathbf{x}_i).$$



Russian Roulette via throughput: At each bounce, continue tracing with survival probability

$$q = \min\left(\max_{1 \leq j \leq d} \beta_j, 1\right),$$

and *divide* β by q if the path survives. This keeps the estimator **unbiased** while early-terminating low-contribution paths.

Connection to pseudocode

The variable α in the path tracing algorithm is the throughput magnitude. It serves both as the Russian Roulette survival threshold and as the divisor that compensates surviving paths, with $\alpha = \max_{1 \leq j \leq d} w_j(l_{next})$ updated each bounce.



Now, we put everything together to summarize the **Monte-Carlo Ray Tracing** or **Path Tracing** algorithm:

1. Generate primary rays similar to Whitted Ray Tracing

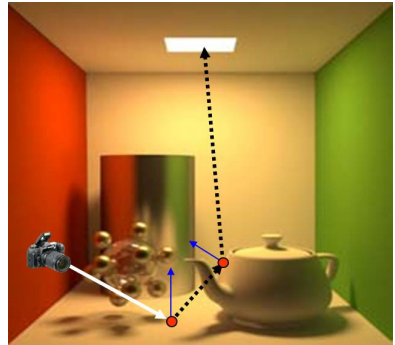


- ▶ More than 1 ray per pixel
→ reconstruction filter needed
- ▶ Only sampling pixel center
- ▶ (Uniform) sampling of the “whole” pixel
→ Anti-Aliasing





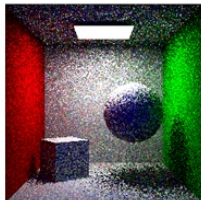
2. Compute intersection with geometry
3. Recursive/iterative evaluation of the Rendering Equation using Monte-Carlo Integration
→ The path is terminated using **Russian Roulette**.



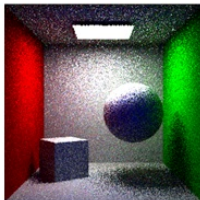
$$L(\mathbf{x}, \omega_o) \approx L_e(\mathbf{x}, \omega_o) + \sum_{i=1}^n \frac{f_{BRDF}(\hat{\omega}_i, \mathbf{x}, \omega_o) \cos(\hat{\theta}_i)}{p(\mathbf{x}, \hat{\omega}_i)} L(h(\mathbf{x}, \hat{\omega}_i), -\hat{\omega}_i)$$



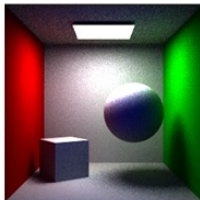
Results:



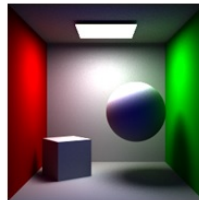
1 sample/pixel



4 samples/pixel



64 samples/pixel



1024 samples/pixel

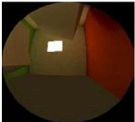
Standard Path Tracing converges slowly. The variance of the Monte Carlo estimator is responsible for the noise artifacts in the rendered image.

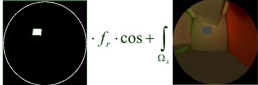


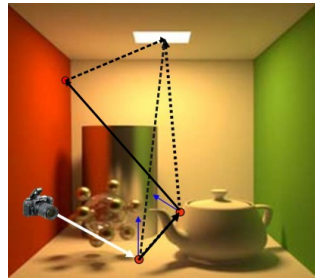
Problem: Usually, scenes contain few and very localized light sources ($L_e > 0$)

- ▶ Small probability to hit light source $\rightarrow$ practically close to zero!
- ▶ **Bad efficiency**

Idea: Separate rendering integral into local and global illumination $\rightarrow$ directly sample local illumination (like Whitted Ray-Tracing)

$$L_o = L_e + \int_{\Omega_x} \cdot f_r \cdot \cos$$


$$L_o = L_e + \int_{\Omega_x}^{\text{local}} \cdot f_r \cdot \cos + \int_{\Omega_x}^{\text{global}} \cdot f_r \cdot \cos$$




At each intersection point the light source is sampled. This modification is called **shadow path tracing**.



Estimator for direct illumination: Sample a point $\mathbf{y}$ on a light source with area PDF $p_A(\mathbf{y})$ and apply the **area-to-solid-angle Jacobian**:

$$\mathbf{L}_{direct}(\mathbf{x}) \approx \frac{\mathbf{f}_{BRDF}(\mathbf{y}-\mathbf{x}, \mathbf{x}, \omega_o) \cdot \mathbf{L}_e(\mathbf{y}) \cdot V(\mathbf{x}, \mathbf{y})}{p_A(\mathbf{y})} \cdot \frac{|\cos(\theta)_x| |\cos(\theta)_y|}{\|\mathbf{y} - \mathbf{x}\|^2}$$

The Jacobian $\frac{|\cos(\theta)_y|}{\|\mathbf{y} - \mathbf{x}\|^2}$ converts area measure $dA(\mathbf{y})$ to solid angle $d\hat{\omega}$ at $\mathbf{x}$. Visibility $V(\mathbf{x}, \mathbf{y}) \in \{0, 1\}$ is evaluated by a **shadow ray**.

Limitation

NEE is highly efficient for large or diffuse light sources, but has *high variance for narrow BRDF lobes* (glossy/specular surfaces): most light samples land outside the lobe and contribute near zero. This complementary failure case motivates combining NEE with BSDF sampling.

Multiple Importance Sampling (MIS)

Problem: BSDF sampling and NEE have complementary failure modes.

BSDF sampling

efficient for smooth / glossy illumination

poor for small or localised lights

NEE (light sampling)

efficient for diffuse surfaces

poor for specular BRDFs

MIS combines both¹ using heuristic weights $w_i(\mathbf{x})$ summing to 1:

$$\langle F \rangle_{\text{MIS}} = \sum_{i=1}^n \frac{1}{n_i} \sum_{j=1}^{n_i} w_i(\mathbf{x}_{i,j}) \frac{f(\mathbf{x}_{i,j})}{p_i(\mathbf{x}_{i,j})}.$$

Balance heuristic (provably near-optimal²):

$$w_i(\mathbf{x}) = \frac{n_i p_i(\mathbf{x})}{\sum_k n_k p_k(\mathbf{x})}.$$

One-sample MIS per bounce in shadow path tracing (one BSDF sample + one NEE sample):

$$L_{\text{direct}} \approx w_{\text{BSDF}} \frac{f_{\text{BRDF}} \cos(\theta) \cdot \mathbf{L}_e}{p_{\text{BSDF}}} + w_{\text{NEE}} \frac{f_{\text{BRDF}} \cos(\theta) \cdot \mathbf{L}_e \cdot V}{p_{\text{NEE}}}$$

Key property



Algorithm Path Tracing

```
1: for each pixel  $p$  do
2:    $L_{pixel} = 0$ 
3:   Create  $m$  primary rays and initialize  $\alpha = 1$ 
4:   for each primary ray  $r_k$  do
5:     if  $\xi > \alpha$  or  $\alpha < 0.001$  (Russian Roulette) then
6:       Stop tracing
7:     end if
8:     Compute next intersection point  $x$ 
9:     View direction  $v = -r_k.dir$ 
10:    Sample light sources:  $L_{local} = \sum_l \frac{f_{BRDF}(l, x, v) \cos(\theta_l)}{p(x, l)} L_e$ 
11:    Recursively sample further direction  $l_{next}$ :  $L_{global} = \frac{w(l_{next}) L_{o_{next}}}{\alpha}$  with
        $w(l_{next}) = \frac{f_{BRDF}(l_{next}, x, v) \cos(\theta_{l_{next}})}{p(x, l_{next})}$  and  $\alpha = \max_{1 \leq j \leq d} w_j(l_{next})$ 
12:     $L_{pixel} = L_{pixel} + L_{local} + L_{global}$ 
13:  end for
14:   $L_{pixel} = L_{pixel} / m$ 
15: end for
```

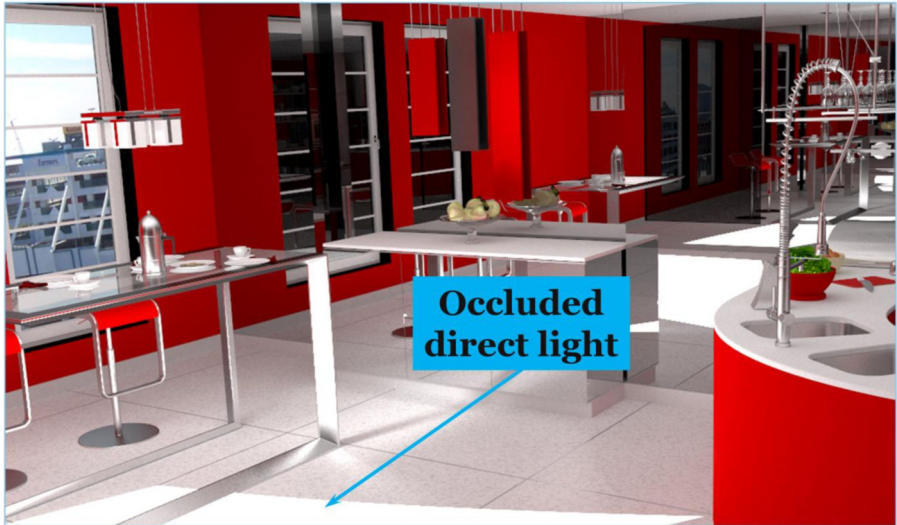


Challenging Cases, example from Krivanek et al., 2014:



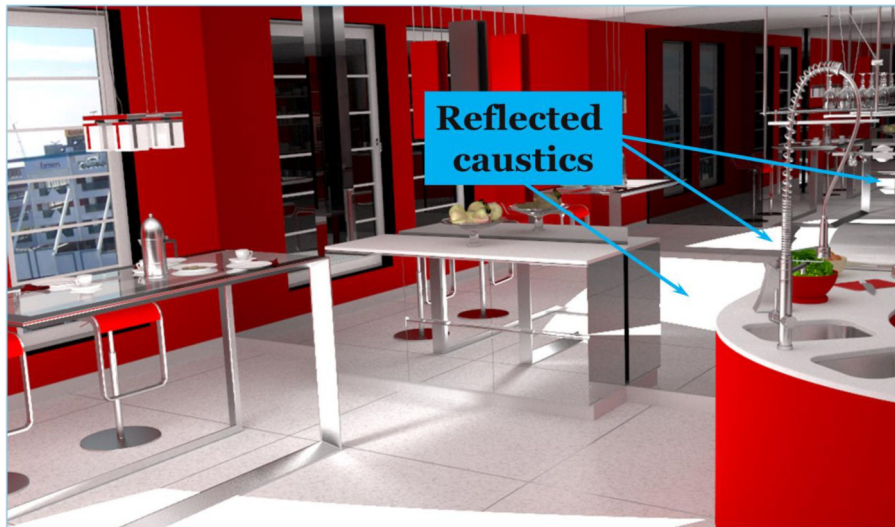


Challenging Cases, example from Krivanek et al., 2014:





Challenging Cases, example from Krivanek et al., 2014:



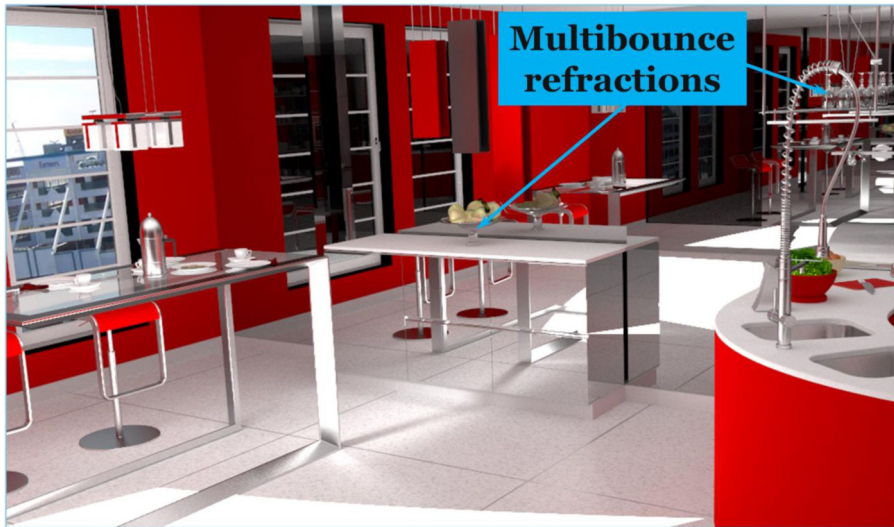


Challenging Cases, example from Krivanek et al., 2014:



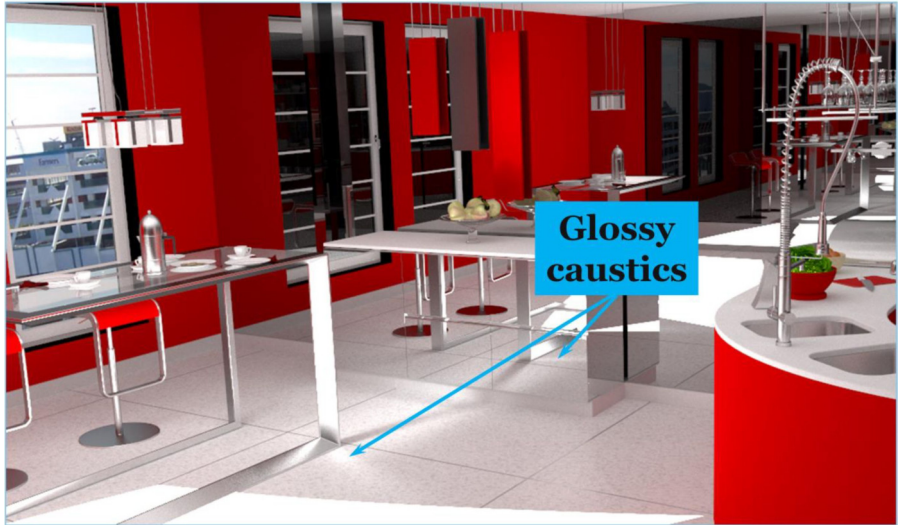


Challenging Cases, example from Krivanek et al., 2014:





Challenging Cases, example from Krivanek et al., 2014:





Result, example from Krivanek et al., 2014:

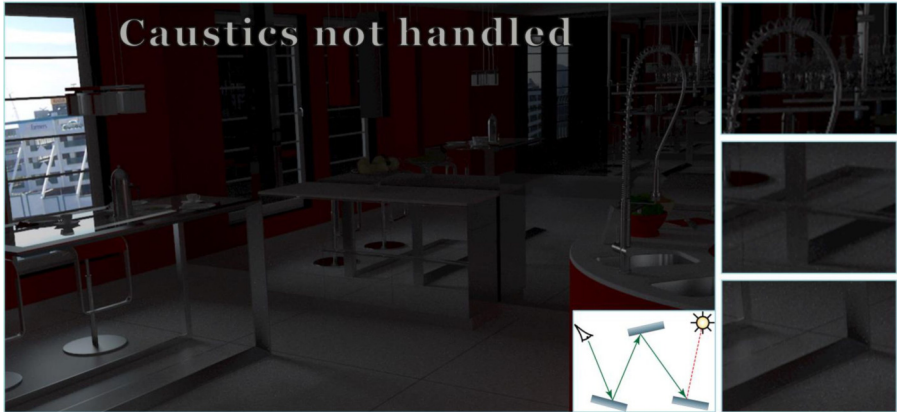


Figure: Result of standard shadow path tracing.



In general **Path Tracing** is not efficient if the probability for important light paths is low:

- ▶ If the light source is large and (or) the illuminated surface has glossy reflections.
- ▶ Light sources hidden. Only the small fraction of light coming from the environment (here all illumination coming from the point light source first comes through the glass. This makes it impossible to directly connect to the light source).
- ▶ Caustics
- ▶ Keyhole effect

Shooting walks are based on the Monte-Carlo solution of the potential equation. They **start at the light source**, go through the scene and try to find the eye. Figure taken from^a.

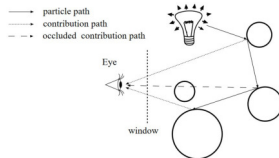
^aL. Szirmay-Kalos. *Monte Carlo Methods in Global Illumination - Script WS 1999/2000*. Vienna University of Technology, 2000.

Light tracing³⁴ is the direct implementation of the Monte-Carlo quadrature of the multi-dimensional formulation of the potential equation.

► Resorting the Path Tracing computations leads to

$$\begin{aligned}
 p(\vec{x}_3) &= \frac{W_e^{(j)}(\hat{x}_0 \rightarrow \hat{x}_1)}{p(\hat{x}_0)} \frac{G(\hat{x}_0 \leftrightarrow \hat{x}_1)\rho(\hat{x}_1)}{p(\hat{x}_1)} \frac{G(\hat{x}_1 \leftrightarrow \hat{x}_2)\rho(\hat{x}_2)}{p(\hat{x}_2)} \frac{G(\hat{x}_2 \leftrightarrow \hat{x}_3)L_e(\hat{x}_3)}{p(\hat{x}_3)} \\
 &= \frac{W_e^{(j)}(\hat{x}_0 \rightarrow \hat{x}_1)G(\hat{x}_0 \leftrightarrow \hat{x}_1)}{p(\hat{x}_0)} \frac{\rho(\hat{x}_1)G(\hat{x}_1 \leftrightarrow \hat{x}_2)}{p(\hat{x}_1)} \frac{\rho(\hat{x}_2)G(\hat{x}_2 \leftrightarrow \hat{x}_3)}{p(\hat{x}_2)} \frac{L_e(\hat{x}_3)}{p(\hat{x}_3)}
 \end{aligned}$$

► Like in the path tracing algorithm when the next direction is determined, BRDF based importance sampling can be applied and combined with Russian-roulette.



³J. Arvo et al. "Backward ray tracing". In: *Developments in ray tracing* 12 (1986), pp. 259–263.

⁴P. Dutré et al. "Monte Carlo light tracing with direct computation of pixel intensities". In: (1993).



Result, example from Krivanek et al., 2014:





Light Tracing copes well with complex indirect illumination on Lambertian surfaces:

- ▶ handles caustics coming through the window quite well
- ▶ Light sources hidden
- ▶ Keyhole effect

It is not efficient if the probability for important light paths is low:

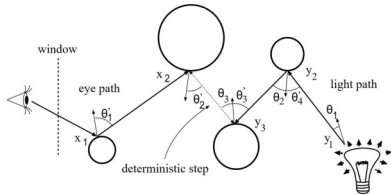
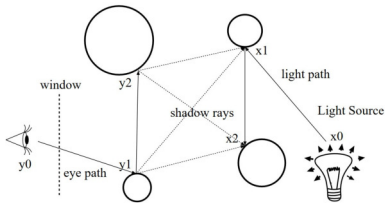
- ▶ it cannot efficiently calculate the caustics that are visible through mirrors and glasses (SDS-paths)
- ▶ Light sources hidden
- ▶ Keyhole effect



Bidirectional Path Tracing



Bidirectional Path Tracing initiates paths at the same time from a selected lightsource and from the viewpoint. To connect both path after some steps two techniques can be used



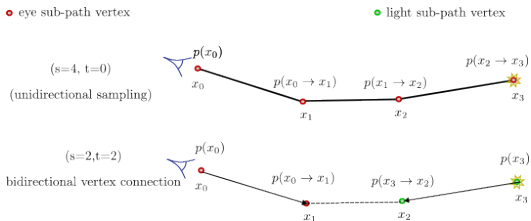
- ▶ all points of the gathering walk are connected to all points of the shooting walk using deterministic rays⁵
- ▶ a single deterministic shadow ray connects the two⁶.

⁵E. P. Lafortune and Y. D. Willems. "Bi-directional path tracing". In: (1993).

⁶E. Veach and L. Guibas. "Bidirectional estimators for light transport". In: *Photorealistic Rendering Techniques*. Springer. 1995, pp. 145–167.

Right: different techniques for sampling a light path, with the corresponding pdf terms associated with each vertex.

For paths with k edges (here $k = 3$) bidirectional path tracing provides $k + 2$ sampling techniques.



Connecting camera and light path: Sampling one sub-path from a light source and another one from the eye, optionally connecting them with an edge. In this case, the conditional vertex pdfs are:

$$p(x_j) = \begin{cases} p(x_{j-1} \rightarrow x_j), & \text{if } x_j \text{ is on a eye sub-path,} \\ p(x_{j+1} \rightarrow x_j), & \text{if } x_j \text{ is on an light sub-path,} \end{cases}$$

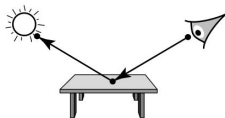
where $p(x_i \rightarrow x_j)$ is the pdf for sampling the direction from x_i to x_j , expressed w.r.t. the area measure. Two special cases, $j = 0$ on a light sub-path and $j = k$ on an eye sub-path, are the pdfs for the starting point of the light and eye sub-paths, respectively.



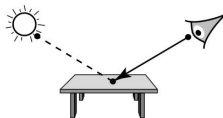
Bidirectional Path Tracing



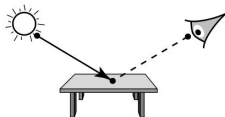
The four bidirectional sampling techniques for paths of length $k = 2^7$.



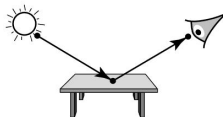
(a) $s = 0, t = 3$



(b) $s = 1, t = 2$



(c) $s = 2, t = 1$



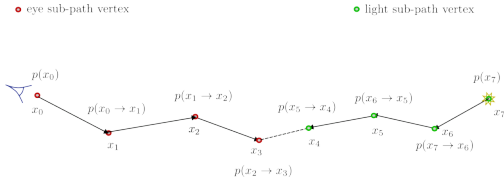
(d) $s = 3, t = 0$

(a) Monte Carlo path tracing with no special handling of light sources, (b) Monte Carlo path tracing with a direct lighting calculation, (c) tracing photons from the light sources and recording an image sample whenever a photon hits a visible surface, and (d) tracing photons and recording an image sample only when photons hit the camera lens. Note that technique (a) can only be used with an area light source, while technique (d) can only be used with a finite-aperture lens.

⁷E. Veach. *Robust Monte Carlo methods for light transport simulation*. Stanford University, 1998.



Bidirectional Path Tracing



Resorting of the Path Tracing Computations lead to:

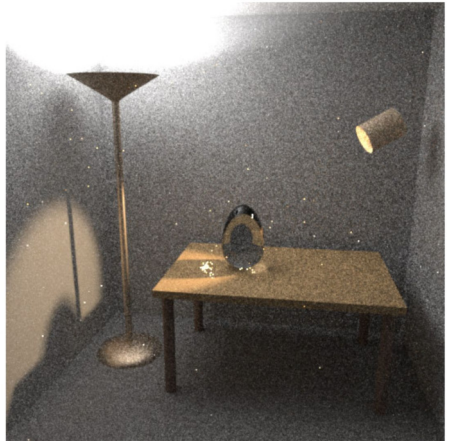
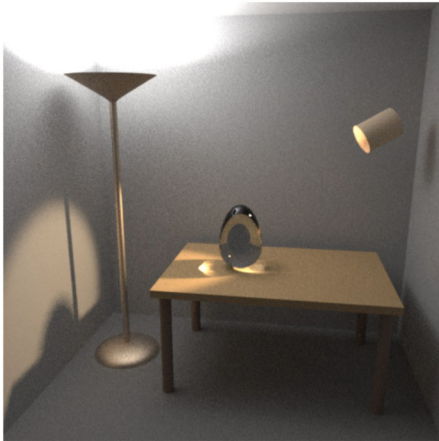
$$\begin{aligned}
 p(\bar{x}_7) &= \frac{W_e^{(j)}(\hat{x}_0 \rightarrow \hat{x}_1)}{p(\hat{x}_0)} \frac{G(\hat{x}_0 \leftrightarrow \hat{x}_1)\rho(\hat{x}_1)}{p(\hat{x}_1)} \frac{G(\hat{x}_1 \leftrightarrow \hat{x}_2)\rho(\hat{x}_2)}{p(\hat{x}_2)} \frac{G(\hat{x}_2 \leftrightarrow \hat{x}_3)\rho(\hat{x}_3)}{p(\hat{x}_3)} \\
 &\quad \frac{G(\hat{x}_3 \leftrightarrow \hat{x}_4)\rho(\hat{x}_4)}{p(\hat{x}_4)} \frac{G(\hat{x}_4 \leftrightarrow \hat{x}_5)\rho(\hat{x}_5)}{p(\hat{x}_5)} \frac{G(\hat{x}_5 \leftrightarrow \hat{x}_6)\rho(\hat{x}_6)}{p(\hat{x}_6)} \frac{G(\hat{x}_6 \leftrightarrow \hat{x}_7)L_e(\hat{x}_7 \rightarrow \hat{x}_6)}{p(\hat{x}_7)} \\
 &= \frac{W_e^{(j)}(\hat{x}_0 \rightarrow \hat{x}_1)}{p(\hat{x}_0)} \frac{G(\hat{x}_0 \leftrightarrow \hat{x}_1)\rho(\hat{x}_1)}{p(\hat{x}_1)} \frac{G(\hat{x}_1 \leftrightarrow \hat{x}_2)\rho(\hat{x}_2)}{p(\hat{x}_2)} \frac{G(\hat{x}_2 \leftrightarrow \hat{x}_3)}{p(\hat{x}_3)} \rho(\hat{x}_3)G(\hat{x}_3 \leftrightarrow \hat{x}_4)\rho(\hat{x}_4) \\
 &\quad \frac{G(\hat{x}_4 \leftrightarrow \hat{x}_5)}{p(\hat{x}_4)} \frac{\rho(\hat{x}_5)G(\hat{x}_5 \leftrightarrow \hat{x}_6)}{p(\hat{x}_5)} \frac{\rho(\hat{x}_6)G(\hat{x}_6 \leftrightarrow \hat{x}_7)}{p(\hat{x}_6)} \frac{L_e(\hat{x}_7 \rightarrow \hat{x}_6)}{p(\hat{x}_7)}
 \end{aligned}$$

Connecting camera and light path

To connect a camera path $(x_0, \dots, x_i)$ with a light path $(x_7, \dots, x_{i+1})$ the contribution of the two pathes have to be multiplied by $\rho(\hat{x}_i)G(\hat{x}_i \leftrightarrow \hat{x}_{i+1})\rho(\hat{x}_{i+1})$.



Result, example from⁸:



Left: Bidirectional path tracing with 25 samples per pixel, Right: Standard path tracing with 56 samples per pixel (both with the same computation time)

⁸E. Veach. *Robust Monte Carlo methods for light transport simulation*. Stanford University, 1998.



Bidirectional Path Tracing



Example weighted contribution that each bidirectional sampling technique $p(s; t)$

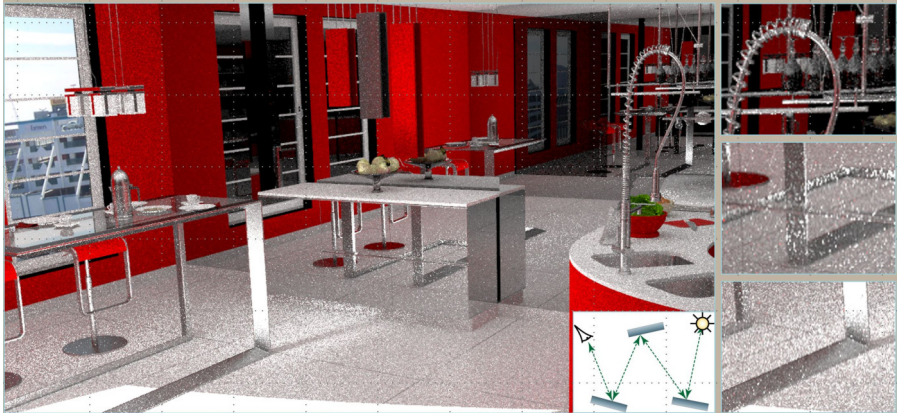


Each row r shows the contributions of the sampling techniques for a particular path length $k = r + 1$ to the image on the previous slide.



Result, example from Krivanek et al., 2014:

Bidirectional Path Tracing

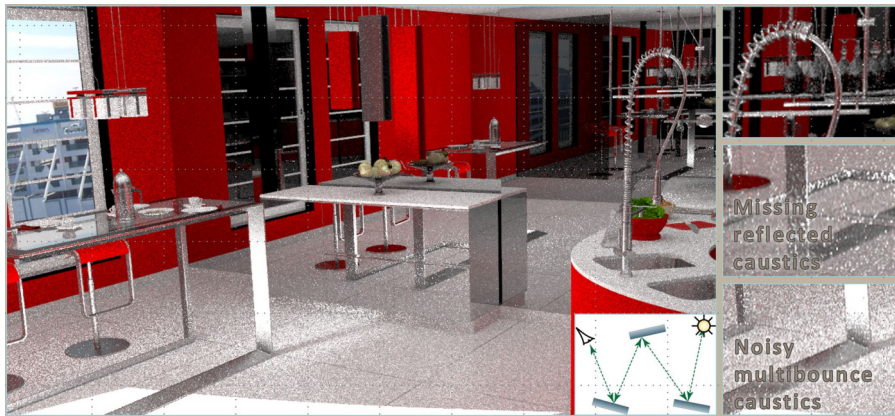




Bidirectional Path Tracing



Result, example from Krivanek et al., 2014:





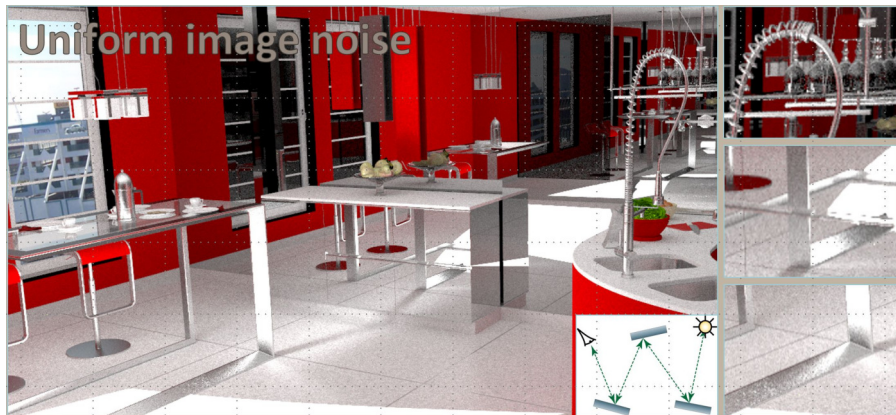
The benefit of BPT is

- ▶ that it works much better for scenes where the light source is "hard to get to" either because it is small or because it requires an unlikely path for it to contribute to the image. Any strong indirect lighting, like in the scene above where most of the scene's illumination is coming from the light source outside the room is a great example.
- ▶ Once we have a full path that connects the camera to a light source we can reduce variance even further by using **MIS** to weigh each possible way the path could have been constructed: from the camera to the light, from the light to the camera, and each possible pair of connection vertices along the way (see MIS slide above).

Problems of BPT

- ▶ Bidirectional path tracing handles almost all possible paths, however some difficult paths are noisy or missing.
- ▶ BDPT has difficulties sampling caustics that go through multiple bounces. The reason is that the mass of such caustics in the path space is small, making it difficult to find them stochastically.
- ▶ The reflected caustics from point light cannot be sampled due to missing connection opportunity (that is, a path edge without singular materials)

Result, example from Krivanek et al., 2014:



Bidirectional Path Tracing can be improved by incorporating ideas from Photon Mapping: light subpath vertices within a small radius r of an eye subpath vertex are **merged** rather than connected, handling SDS paths and multi-bounce caustics that BDPT misses. Details in the next slide set.



- ▶ For the OMC-based methods, there are a lot of heuristic approaches that try to improve the calculation of certain individual phenomena and that perform well on certain classes of scenes (typically under severe restrictions).
- ▶ However, these approaches are difficult to use in practice because the integration space is multidimensional and complex: in such a space, each assumption made by a heuristic is sooner or later violated.
- ▶ The only method that ensures robustness in this case is **Multiple Importance Sampling (MIS)**. However the more a method is robust, the more intricate is its implementation and testing and it is slower on the average.
- ▶ As a result, different methods should be used in different cases, which makes light transport simulation problem extremely difficult for both developers and users of rendering systems.
- ▶ Currently, the most advanced MIS-based approach is **CMIS** (Continuous MIS)¹⁰, which extends MIS to a continuum of sampling strategies represented by parametric functions, improving convergence in certain cases. However, the problem of optimal strategy selection in CMIS remains open.

¹⁰J. Krivanek et al. "Recent advances in light transport simulation: some theory and a lot of practice". In: *ACM SIGGRAPH 2014 Courses* (2014), pp. 1–6.



- [1] J. Arvo et al. “Backward ray tracing”. In: *Developments in ray tracing* 12 (1986), pp. 259–263.
- [2] P. Dutré, E. P. Lafortune, and Y. D. Willems. “Monte Carlo light tracing with direct computation of pixel intensities”. In: (1993).
- [3] I. Georgiev, J. Krivanek, T. Davidovic, and P. Slusallek. “Light transport simulation with vertex connection and merging.”. In: *ACM Trans. Graph.* 31.6 (2012), pp. 192–1.
- [4] J. T. Kajiya. “The rendering equation”. In: *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*. 1986, pp. 143–150.
- [5] J. Krivanek, A. Keller, I. Georgiev, A. S. Kaplanyan, M. Fajardo, M. Meyer, J.-D. Nahmias, O. Karljik, and J. Canada. “Recent advances in light transport simulation: some theory and a lot of practice”. In: *ACM SIGGRAPH 2014 Courses* (2014), pp. 1–6.
- [6] E. P. Lafortune and Y. D. Willems. “Bi-directional path tracing”. In: (1993).
- [7] L. Szirmay-Kalos. *Monte Carlo Methods in Global Illumination - Script WS 1999/2000*. Vienna University of Technology, 2000.



- [8] E. Veach. *Robust Monte Carlo methods for light transport simulation*. Stanford University, 1998.
- [9] E. Veach and L. Guibas. "Bidirectional estimators for light transport". In: *Photorealistic Rendering Techniques*. Springer. 1995, pp. 145–167.